

Curso de Go

Aula 13 - Pacotes, visibilidade e organização em Go

Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 **C/B** [código_] **BRAZUCA** [3]





Nessa Aula nos Vamos Ver

- **O que são pacotes em Go**
- **Como organizar arquivos e pastas**
- **Como importar pacotes**
- **Como funcionam nomes exportados**
- **Por que letras maiúsculas importam**
- **Como separar código reutilizável**
- **Como estruturar projetos pequenos e médios**
- **Boas práticas para evitar bagunça no código**
- **Parte Prática**



O que é um pacote em Go?

- Em Go, todo arquivo pertence a um pacote.
- Um pacote é uma forma de agrupar código relacionado. Ele pode conter funções, structs, interfaces, constantes e variáveis.
- Todo arquivo Go começa com:

```
package main
```

- **Exemplo**

```
package main
```

```
import "fmt"
```

```
func main() {  
    fmt.Println("Olá, Go!")  
}
```



O que é um pacote em Go?

- O pacote `main` tem um significado especial: ele indica que o programa pode ser executado.
- Quando usamos:

```
go run main.go
```

- O Go procura a função:

```
func main()
```

- Ela é o ponto de entrada do programa.



Pacote main e pacotes reutilizáveis

- O pacote main é usado para criar programas executáveis.

- Exemplo:

```
package main

import "fmt"

func main() {
    fmt.Println("Sistema iniciado")
}
```

- Mas nem todo pacote precisa ser executável.
- Podemos criar pacotes reutilizáveis, por exemplo:

```
package matematica

func Somar(a int, b int) int {
    return a + b
}
```

- Ele serve para ser usado por outros arquivos ou outros projetos.



Estrutura básica de um projeto Go

- Um projeto simples pode ter esta estrutura:

```
meu-projeto/  
├── go.mod  
├── main.go  
└── calculadora/  
    └── calculadora.go
```

- O arquivo go.mod define o módulo do projeto.

```
go mod init exemplo.com/meu-projeto
```

- Isso cria:

```
module exemplo.com/meu-projeto
```

```
go 1.22
```



Criando um pacote próprio

- **Arquivo:**

`calculadora/calculadora.go`

- **Código**

```
package calculadora
```

```
func Somar(a int, b int) int {  
    return a + b  
}
```

```
func Subtrair(a int, b int) int {  
    return a - b  
}
```



Criando um pacote próprio

- Agora podemos importar esse pacote no main.go:

```
package main

import (
    "fmt"
    "exemplo.com/meu-projeto/calculadora"
)

func main() {
    fmt.Println(calculadora.Somar(8, 2))
    fmt.Println(calculadora.Subtrair(8, 2))
}
```

- Resultado

10
6

Nota Importante: A pasta se chama **calculadora** e o pacote também se chama **calculadora**. Isso é uma prática comum e recomendada!



Regras de exportação em Go

- Em Go, a visibilidade é controlada pela primeira letra do nome.
- Se começa com letra maiúscula, é exportado.

```
func Somar(a int, b int) int {  
    return a + b  
}
```

- Essa função pode ser usada fora do pacote.
- Se começa com letra minúscula, não é exportado.
- Exemplo:

```
func validarNumero(n int) bool {  
    return n >= 0  
}
```

- Essa função só pode ser usada dentro do próprio pacote.



Regras de exportação em Go

- **Exemplo completo:**

```
package calculadora
```

```
func Somar(a int, b int) int {  
    return a + b  
}
```

```
func validarNumero(n int) bool {  
    return n >= 0  
}
```

- **Fora do pacote, podemos chamar:**

```
calculadora.Somar(1, 2)
```

- **Mas não podemos chamar:**

```
calculadora.validarNumero(10)
```



Exportação com structs

- A regra da letra maiúscula também vale para structs e campos.

```
package usuario
```

```
type Usuario struct {  
    Nome string  
    Email string  
    senha string  
}
```

- No caso acima Nome e Email são exportados (maiúsculos), senha, não (minúscula)



Encapsulamento simples com construtor

- Uma boa prática é esconder campos internos e criar funções para controlar o acesso.

```
package usuario

type Usuario struct {
    Nome string
    Email string
    senha string
}

func NovoUsuario(nome string, email string, senha string) Usuario
{
    return Usuario{
        Nome: nome,
        Email: email,
        senha: senha,
    }
}

func (u Usuario) VerificarSenha(s string) bool {
    return u.senha == s
}
```



Encapsulamento simples com construtor

- **Uso prático no main.go**

```
package main

import (
    "fmt"
    "exemplo.com/meu-projeto/usuario"
)

func main() {
    u := usuario.NovoUsuario("Carlos", "carlos@email.com",
    "123456")

    fmt.Println(u.Nome)
    fmt.Println(u.Email)
    fmt.Println(u.VerificarSenha("123456"))
}
```

- Dessa forma, o campo senha fica protegido.
- O usuário do pacote não acessa diretamente o dado sensível.



Organização para reuso

- **Uma boa organização separa responsabilidades.**
- **Exemplo:**

```
loja/
├── go.mod
├── main.go
├── produto/
│   └── produto.go
├── pedido/
│   └── pedido.go
└── pagamento/
    └── pagamento.go
```

- **Cada pacote cuida de uma parte do sistema.**
- **Isso facilita manutenção e reaproveitamento.**



Organização para reuso - exemplo

- **Pacote produto:**

```
package produto

type Produto struct {
    Nome string
    Preço float64
}

func NovoProduto(nome string, preco
float64) Produto {
    return Produto{
        Nome: nome,
        Preço: preco,
    }
}
```

- **Pacote pedido:**

```
package pedido

import "exemplo.com/loja/produto"

type Pedido struct {
    Itens []produto.Produto
}

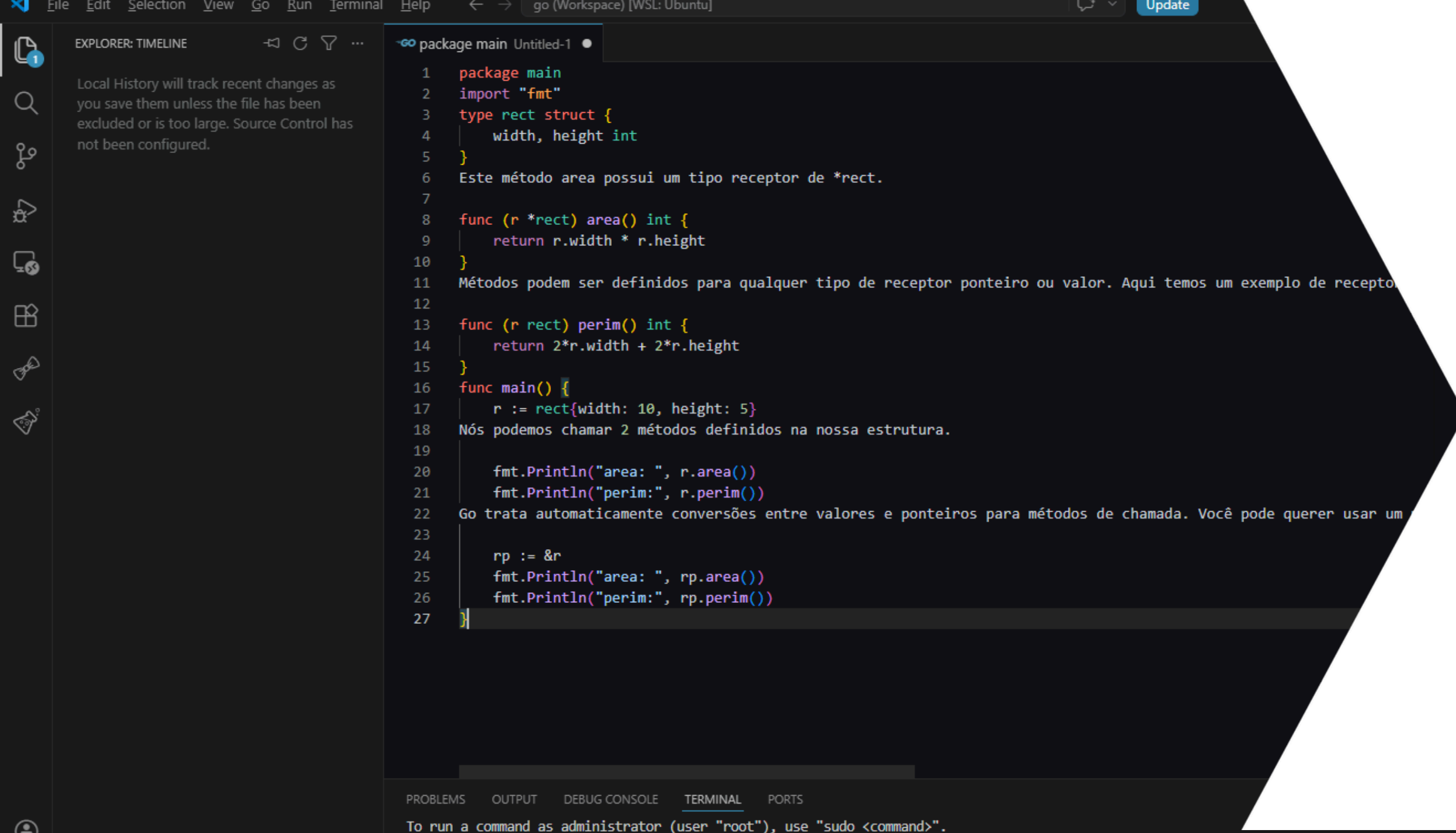
func (p *Pedido) Adicionar(item
produto.Produto) {
    p.Itens = append(p.Itens, item)
}
```



Boas práticas

- Evite criar pacotes genéricos demais, como: utils, helpers, common
- Prefira nomes que indiquem responsabilidade: usuario, pedido, pagamento, relatorio, produto
- Um pacote deve ter uma responsabilidade clara
- Use nomes curtos e objetivos
- Exporte apenas o que outros pacotes precisam usar
- Mantenha funções auxiliares privadas
- Evite dependências circulares
- Organize o código pensando em manutenção
- Prefira pacotes pequenos e fáceis de testar

IMPORTANTE: Pacotes em Go ajudam a organizar o sistema, controlar visibilidade e permitir reuso de código com clareza!



```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de receptor
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

NÚCLEA
ACADEMY

Parte Prática

Praticar Pacotes, visibilidade e organização em Go